

Inside the Pico

Del código a la memoria, los registros y el mundo físico: cómo una línea de programa termina cambiando el voltaje de un pin.

CÓDIGO



CPU



BUS



REGISTRO



GPIO



3.3 V

Laboratorio de Elementos Programables I

Prof. Luis Eduardo Arenas Deseano

01

¿Qué hay dentro del microcontrolador?

COMPUTADORA GENERAL

Sistema operativo

Procesos

GB de RAM

Archivos grandes

MICROCONTROLADOR (LA PICO)

Sin Linux

Memoria limitada

Control de pines

Tiempo real ligero

| La ingeniería embebida consiste en hacer mucho con muy poca memoria.

RP2040 vs RP2350: misma familia, más capacidad

CARACTERÍSTICA	RP2040 / PICO W	RP2350 / PICO 2 W
CPU	2× Arm Cortex-M0+	2× Arm Cortex-M33 o 2× Hazard3 RISC-V
SRAM	264 KB	520 KB
PIO	8 state machines	12 state machines
Board target	pico_w	pico2_w
Enfoque	aprendizaje + IoT básico	más memoria + más seguridad + más potencia

Para nuestros ejemplos usamos abstracciones: así el mismo concepto funciona en ambas placas.

Mapa de memoria: la Pico como una ciudad

REGIÓN

Colonia o zona de la ciudad: Flash, SRAM, periféricos, SIO.

DIRECCIÓN

Calle y número:
0x20000000,
0x40000000,
0xD0000000.

REGISTRO

Departamento específico donde vive una variable de hardware. Bit = interruptor dentro del departamento.

Variable → Memoria → Dirección → Hardware `int x = 42; → &x = 0x2000xxxx`

DIRECCIÓN	REGIÓN	CONTIENE
0x00000000	ROM	Código de arranque
0x10000000	XIP / FLASH	Programa y constantes
0x20000000	SRAM	Variables en ejecución
0x40000000	APB	Periféricos
0x50000000	AHB	Periféricos rápidos
0xD0000000	SIO	GPIO rápido y CPU local

Una dirección no es "solo un número": puede apuntar a memoria o a un registro de hardware.

04

SRAM · Flash · Stack · Heap

FLASH / XIP

Programa, constantes, main.py / firmware.

SRAM

Variables, buffers, objetos vivos.

STACK

Variables locales, llamadas a funciones — crece y decrece rápido.

HEAP

Memoria dinámica — listas, bytearray, malloc/free.

| Manejo de memoria = saber cuánto tienes, dónde se guarda y cuándo se libera.

05

Registros y GPIO

Un registro es como una variable, pero vive en una dirección conectada a hardware: cambiar un bit puede cambiar voltaje, temporizadores, PWM, UART, ADC o interrupciones.

0

GPIO LOW

≈ 0 V

1

GPIO HIGH

≈ 3.3 V

registro → GPIO → bit del pin → voltaje físico

06

MicroPython vs C/C++: del programa al silicio

MICROPYTHON

main.py



Intérprete / VM



Módulo machine



Hardware

C/C++

main.c



Compilador



Código máquina



Hardware

El SDK evita escribir direcciones de registros manualmente: nos da funciones y estructuras portables.

07

Evidencia: Wokwi vs físico

SÍ USAR WOKWI PARA

GPIO

Lógica

Serial

PWM básico

Depuración

NO USARLO PARA AFIRMAR

Rendimiento real

Consumo de energía

Tiempos exactos

Comportamiento
completo RP2350

VALIDACIÓN FINAL

Placa física

Serial real

Fotos/video

Mediciones reales

```
1 import gc, os, machine
2 print(os.uname())
3 print('freq:', machine.freq())
4 gc.collect()
5 print('libre inicial:', gc.mem_free())
6 buffer = bytearray(10000)
7 print('después de reservar:', gc.mem_free())
8 del buffer
9 gc.collect()
10 print('después de liberar:', gc.mem_free())
```

```
buffer = bytearray(10000)
```

```
(sysname='rp2', nodename='rp2', release='1.28.0', version='v1.28.0 on 2026-04-06 (GNU 14.2.0
MinSizeRel)', machine='Raspberry Pi Pico with RP2040')
freq: 125000000
libre inicial: 241168
después de reservar: 231136
después de liberar: 241152
MicroPython v1.28.0 on 2026-04-06; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>
```

```
1 import gc, os, machine
2 print(os.uname())
3 print('freq:', machine.freq())
4 gc.collect()
5 print('libre inicial:', gc.mem_free())
6 buffer = bytearray(20000)
7 print('después de reservar:', gc.mem_free())
8 del buffer
9 gc.collect()
10 print('después de liberar:', gc.mem_free())
```

```
buffer = bytearray(20000)
```

```
(sysname='rp2', nodename='rp2', release='1.28.0', version='v1.28.0 on 2026-04-06 (GNU 14.2.0
MinSizeRel)', machine='Raspberry Pi Pico with RP2040')
freq: 125000000
libre inicial: 241168
después de reservar: 221136
después de liberar: 241152
MicroPython v1.28.0 on 2026-04-06; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>>
```

Wokwi ayuda a aprender y depurar, pero la validación final se hace con hardware real.

Jonathan Hernández Lazcano

Cuenta 200417

Universidad Iberoamericana de Puebla

Laboratorio de Elementos Programables I

Sesión 02 • Conceptos básicos MCU